

API- und Schnittstellenarchitektur – EMS-Forecast

1. Ziel der Architektur

Ziel der API- und Schnittstellenarchitektur ist es, die Datenflüsse im System klar zu trennen. Externe Datenquellen dienen zur Anreicherung der zentralen Datenbasis. Das Frontend greift hingegen ausschließlich auf eine eigene Backend-API zu und kommuniziert nicht direkt mit externen APIs oder dem Prognosemodell.

Das System soll Einsatzwahrscheinlichkeiten für definierte Gebiete in Oberösterreich berechnen und diese in einer interaktiven Heatmap darstellen. Die Berechnung basiert auf historischen Einsatzdaten sowie zusätzlichen Einflussfaktoren wie Wetter, Verkehr, Demografie, Feiertagen und zeitlichen Mustern.

2. Grundsätzlicher Systemaufbau

```
Externe Datenquellen / APIs  
Wetter, Verkehr, Demografie, Feiertage, Events, Geodaten  
↓  
Importer / ETL-Prozesse  
↓  
Zentrale Datenbank  
Einsatzdaten, Gebiete, Kontextdaten, Prognosewerte  
↓  
Prognosemodell / Prediction-Service  
↓  
Berechnete Einsatzwahrscheinlichkeiten  
↓  
Eigene Backend-API (.NET)  
↓  
Angular Frontend  
Heatmap, Zeitraum-Auswahl, Einsatztyp-Ansicht
```

3. Trennung der Schnittstellenbereiche

Die Architektur unterscheidet zwischen zwei wesentlichen Schnittstellenbereichen:

3.1 Externe Datenquellen und Import-Schnittstellen

Dieser Bereich umfasst alle externen APIs und Datenquellen, die für die Datenbasis und das Prognosemodell relevant sind.

Beispiele:

- Wetterdaten, z. B. Temperatur, Niederschlag, Wind, Luftfeuchtigkeit
- Verkehrsdaten, z. B. Stau, Verkehrslage, Straßenbelastung
- demografische Daten, z. B. Bevölkerung, Altersgruppen, Geschlecht
- Kalenderdaten, z. B. Feiertage, Ferien, Events

- Geodaten, z. B. PLZ, Gemeinden, Gebiete, GeoJSON-Grenzen

Diese Daten werden nicht direkt vom Frontend verwendet. Sie werden über Importer, Skripte oder geplante Jobs gesammelt, bereinigt, vereinheitlicht und in der zentralen Datenbank gespeichert.

3.2 Eigene Backend-API für das Frontend

Das Frontend kommuniziert ausschließlich mit der eigenen Backend-API. Diese API stellt fertige, für die Visualisierung geeignete Daten bereit.

Die Backend-API kapselt:

- Zugriff auf die Datenbank
- Zugriff auf gespeicherte Prognosewerte
- optionalen Aufruf eines Prediction-Services
- Aufbereitung der Daten für die Heatmap
- Rückgabe der Wahrscheinlichkeiten je Gebiet

Das Frontend muss dadurch keine Kenntnisse über externe Datenquellen, Modelllogik oder Datenbankstrukturen besitzen.

4. Rolle der Datenbank

Die zentrale Datenbank ist der Kern des Systems. In ihr werden alle relevanten Daten strukturiert abgelegt.

Mögliche Datenbereiche:

- Gebiete und PLZ-Zuordnungen
- historische Einsatzdaten
- synthetische Einsatzdaten
- Wetterdaten je Gebiet und Zeitpunkt
- Verkehrsdaten je Gebiet und Zeitpunkt
- demografische Daten je Gebiet oder Bezirk
- Feiertage, Ferien und Events
- berechnete Prognosewerte

Das Prognosemodell arbeitet primär mit dieser Datenbank. Auch aktuelle Daten wie Wetter oder Verkehr werden zuerst importiert oder über definierte Prozesse verfügbar gemacht, bevor sie in die Berechnung einfließen.

5. Rolle des Prognosemodells

Das Prognosemodell berechnet auf Basis der vorhandenen Daten Einsatzwahrscheinlichkeiten für die definierten Gebiete.

Das Modell kann technisch unterschiedlich eingebunden werden:

- als Python-Skript
- als geplanter Worker/Job

- als eigener interner Prediction-Service
- als separat laufende Modell-API
- später optional direkt über das Backend orchestriert

Für den Prototyp ist nicht entscheidend, ob das Modell dauerhaft als eigene API läuft. Wichtig ist, dass das Ergebnis in einer klaren Struktur verfügbar ist.

Mögliche Ergebnisdaten:

- Gebiet-ID
 - Prognosezeitraum
 - berechnete Einsatzwahrscheinlichkeit
 - Risikostufe für die Heatmap
 - optional: wahrscheinlichste Einsatztypen
 - optional: Modellversion oder Berechnungszeitpunkt
-

6. Rolle des Frontends

Das Angular-Frontend dient zur Visualisierung der Ergebnisse. Es soll keine Rohdaten verarbeiten und keine externen APIs direkt abfragen.

Die Hauptaufgaben des Frontends sind:

- GeoJSON-Karte anzeigen
- Gebiete anhand der Prognosewerte einfärben
- Prognosezeitraum auswählen
- optional Einsatztyp auswählen
- Detailinformationen zu einem Gebiet anzeigen
- historische Ansicht oder Vergleichsansicht darstellen

Nicht Aufgabe des Frontends:

- Wetterdaten selbst abrufen
 - Verkehrsdaten selbst abrufen
 - Modelllogik ausführen
 - Rohdaten bereinigen
 - externe Datenquellen integrieren
-

7. Sinnvolle Frontend-Abfragen

Das Frontend benötigt nur wenige fachlich sinnvolle Auswahlmöglichkeiten.

7.1 Prognosehorizont

Der wichtigste Parameter ist der gewünschte Prognosezeitraum.

Beispiele:

- nächste 1 Stunde

- nächste 24 Stunden
- nächste 7 Tage
- benutzerdefinierter Zeitraum

Je weiter der Prognosezeitraum in der Zukunft liegt, desto stärker muss das Backend bzw. Modell entscheiden, ob aktuelle Prognosedaten oder historische Durchschnittswerte verwendet werden.

Beispiel:

- 1 Stunde: aktuelle Wetter- und Verkehrsdaten können relevant sein
- 24 Stunden: Wetterprognosen können verwendet werden
- 7 Tage: Wetterprognosen sind eingeschränkt belastbar, historische Muster werden wichtiger
- längerer Zeitraum: vor allem historische, saisonale und demografische Muster relevant

7.2 Einsatztyp

Optional kann das Frontend eine Auswahl nach Einsatztyp anbieten.

Beispiele:

- alle Einsätze
- Verkehrsunfall
- internistischer Notfall
- Sturz
- Atemnot
- Herz-Kreislauf
- psychische Gründe
- Intoxikation
- sonstige Einsätze

Diese Auswahl ist fachlich sinnvoll, wenn das Modell Wahrscheinlichkeiten auch nach Einsatzkategorien berechnen kann.

7.3 Ansicht

Optional kann zwischen verschiedenen Darstellungen gewechselt werden.

Beispiele:

- Prognoseansicht
- historische Vergleichsansicht
- kombinierte Ansicht

8. Beispielhafte Backend-API für das Frontend

8.1 Prognose für Heatmap abrufen

```
GET /api/forecast?from=2026-06-18T12:00:00&to=2026-06-19T12:00:00&incidentType=all
```

Beispielantwort:

```
[
  {
    "gebietId": 1,
    "gebietName": "Linz + Urfahr",
    "probability": 0.74,
    "riskLevel": 4,
    "topIncidentTypes": [
      {
        "type": "Internistisch",
        "probability": 0.31
      },
      {
        "type": "Sturz",
        "probability": 0.22
      }
    ]
  },
  {
    "gebietId": 2,
    "gebietName": "Wels",
    "probability": 0.41,
    "riskLevel": 3,
    "topIncidentTypes": [
      {
        "type": "Verkehrsunfall",
        "probability": 0.18
      },
      {
        "type": "Internistisch",
        "probability": 0.16
      }
    ]
  }
]
```

8.2 Detaildaten für ein Gebiet abrufen

```
GET /api/forecast/areas/1?from=2026-06-18T12:00:00&to=2026-06-19T12:00:00
```

Mögliche Antwort:

```
{
  "gebietId": 1,
  "gebietName": "Linz + Urfahr",
  "forecastPeriod": {
    "from": "2026-06-18T12:00:00",
```

```
    "to": "2026-06-19T12:00:00"
  },
  "probability": 0.74,
  "riskLevel": 4,
  "expectedIncidents": 18,
  "topIncidentTypes": [
    {
      "type": "Internistisch",
      "probability": 0.31
    },
    {
      "type": "Sturz",
      "probability": 0.22
    },
    {
      "type": "Atemnot",
      "probability": 0.14
    }
  ],
  "modelInfo": {
    "calculatedAt": "2026-06-18T10:30:00",
    "modelVersion": "prototype-1"
  }
}
```

8.3 Unterstützte Einsatztypen abrufen

```
GET /api/incident-types
```

Beispielantwort:

```
[
  "all",
  "traffic_accident",
  "internal_emergency",
  "fall",
  "respiratory_distress",
  "cardiovascular",
  "psychological",
  "intoxication",
  "other"
]
```

9. Bewertung der Architektur

Diese Architektur ist für den Prototyp sinnvoll, weil sie klare Verantwortlichkeiten schafft.

Vorteile:

- Das Frontend bleibt einfach und übersichtlich.
 - Externe APIs sind nicht direkt vom Frontend abhängig.
 - Änderungen an Datenquellen betreffen nicht automatisch die Benutzeroberfläche.
 - Die Modelllogik bleibt vom Frontend getrennt.
 - Die Backend-API kann gezielt für die Heatmap optimiert werden.
 - Das System bleibt später leichter erweiterbar.
-

10. Ergebnis für den Sprint

Für diesen Sprint sollte das Ziel nicht sein, alle Schnittstellen vollständig umzusetzen. Sinnvoller ist eine klare technische und fachliche Dokumentation.

Konkrete Sprint-Ergebnisse:

1. Dokumentation der Schnittstellenarchitektur
 2. Entscheidung: Frontend kommuniziert ausschließlich mit eigener Backend-API
 3. Definition erster möglicher Backend-Endpunkte für die Heatmap
 4. Festlegung der relevanten Frontend-Parameter
 5. Vorbereitung eines Angular-Prototyps mit GeoJSON-Karte und Mock-Prognosedaten
-

11. Zusammenfassung

Das System besitzt im Kern zwei unterschiedliche Schnittstellenbereiche.

Erstens gibt es externe Datenquellen und Import-Schnittstellen, über die Wetter-, Verkehrs-, Demografie-, Kalender- und Geodaten in die zentrale Datenbank gelangen.

Zweitens gibt es eine eigene Backend-API, die dem Angular-Frontend fertige Prognosewerte bereitstellt. Das Frontend erhält keine Rohdaten und spricht keine externen APIs direkt an.

Die zentrale Frage des Frontends lautet daher nicht, welche externen Datenquellen verwendet werden, sondern welcher Prognosezeitraum und optional welcher Einsatztyp visualisiert werden soll.